# Data and Artificial Intelligence
# Cyber Shujaa Program

## Week 10 Assignment
## Deep Learning

**Student Name:** Rodney Roy Gitonga
**Student ID:** CS-DA03-26025

## Contents

Contents ..... 1
Introduction ..... 1
Tasks Completed ..... 2
Step 1: Data Loading & Exploration ..... 2
Step 2: Preprocessing ..... 3
Step 3: Model Architecture ..... 3
Step 4: Training ..... 4
Step 4: Evaluation Results ..... 5
Step 4: Training Curves ..... 5
Step 4: Confusion Matrix & Classification Report ..... 6
Step 4: Model Saving ..... 6
Link to Code: ..... 6
https://colab.research.google.com/drive/1VPLLOZLE5dVw5G1FvtMcT4mJpTh9t3A9?usp=sharing ... 6
Conclusion ..... 6

## Introduction

This project implements a feed-forward Artificial Neural Network (ANN) using TensorFlow/Keras to classify handwritten digits from the MNIST dataset. The model demonstrates core deep learning concepts including data preprocessing, model architecture design, regularization with Dropout, training

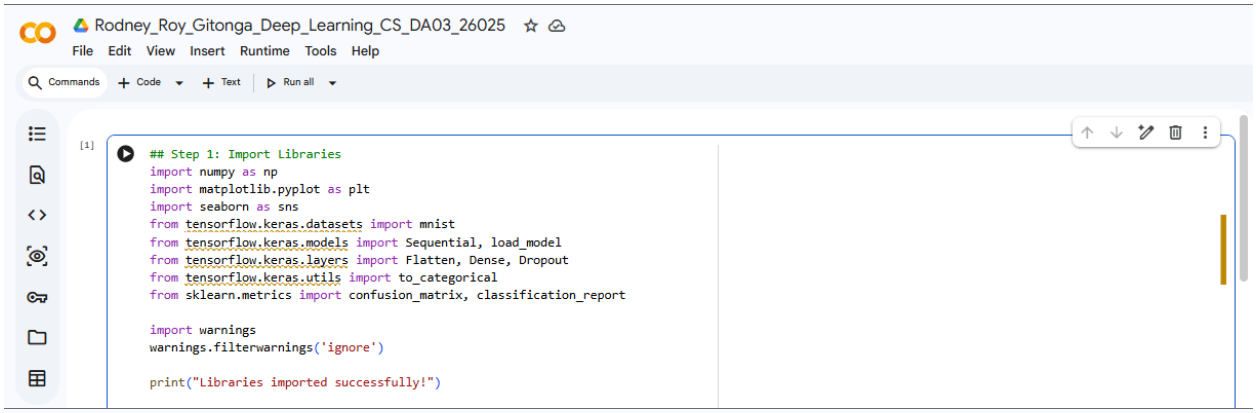
with validation split, and comprehensive evaluation using accuracy, confusion matrix, and classification metrics.

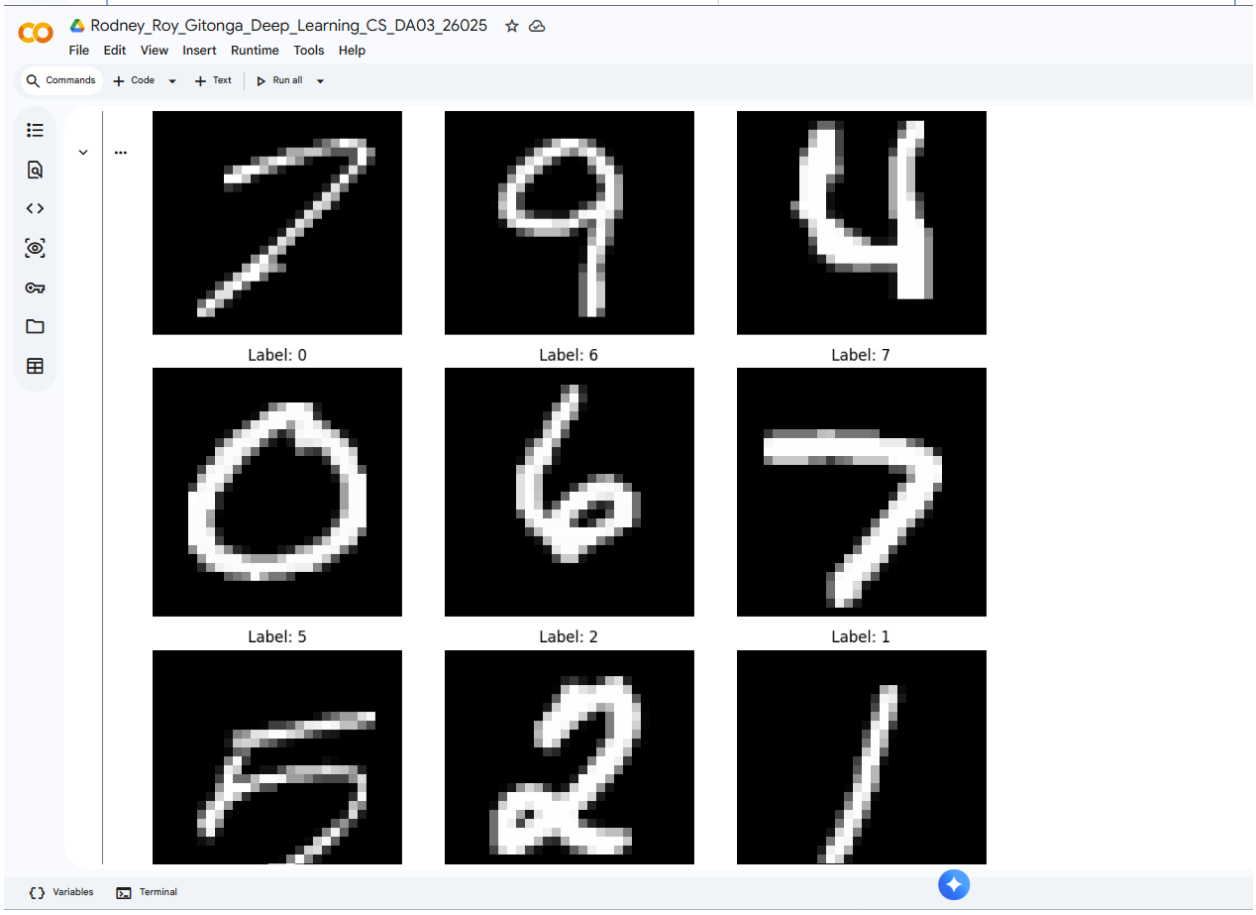
# Tasks Completed

## Step 1: Data Loading & Exploration

1. I loaded 60,000 training and 10,000 test grayscale 28×28 images.

2. Visualized 9 random samples with their true labels.



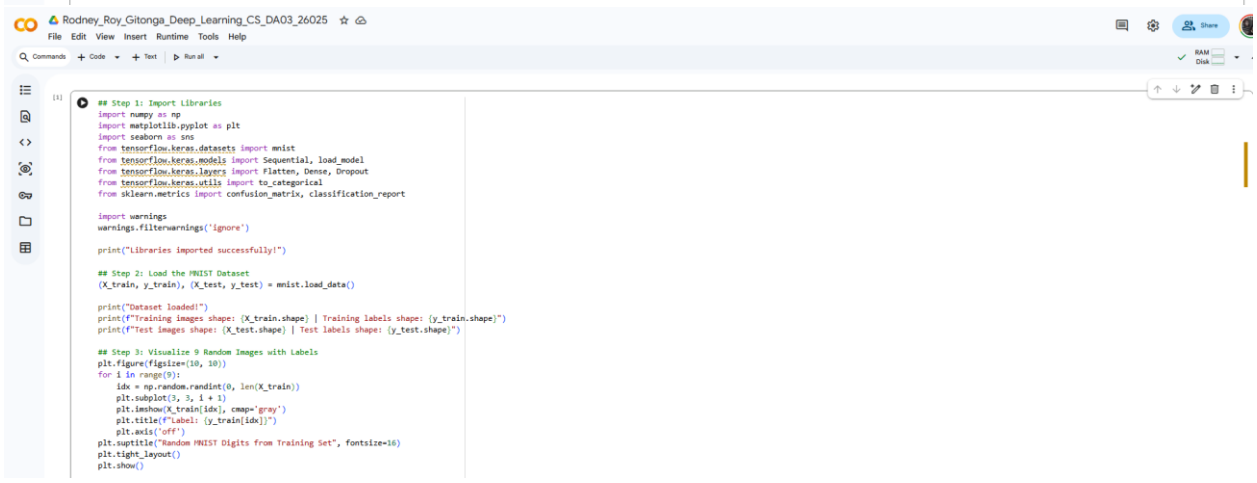
## Step 2: Preprocessing

1. Normalized pixel values from [0-255] to [0-1].
2. Converted integer labels to one-hot encoded vectors (10 classes).

```
Preprocessing completed:
Pixel range after normalization: [0.0, 1.0]
One-hot encoded labels shape (train): (60000, 10)
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 128) | 100,480 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 64) | 8,256 |
| dropout_1 (Dropout) | (None, 64) | 0 |
| dense_2 (Dense) | (None, 10) | 650 |

```
Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
```



```
## Step 1: Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix, classification_report

import warnings
warnings.filterwarnings('ignore')

print("Libraries imported successfully!")

## Step 2: Load the MNIST Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Dataset loaded!")
print(f"Training images shape: {X_train.shape} | Training labels shape: {y_train.shape}")
print(f"Test images shape: {X_test.shape} | Test labels shape: {y_test.shape}")

## Step 3: Visualize 9 Random Images with Labels
plt.figure(figsize=(10, 10))
for i in range(9):
    idx = np.random.randint(0, len(X_train))
    plt.subplot(3, 3, i + 1)
    plt.imshow(X_train[idx], cmap='gray')
    plt.title(f"Label: {y_train[idx]}")
    plt.axis('off')
plt.suptitle("Random MNIST Digits from Training Set", fontsize=16)
plt.tight_layout()
plt.show()
```

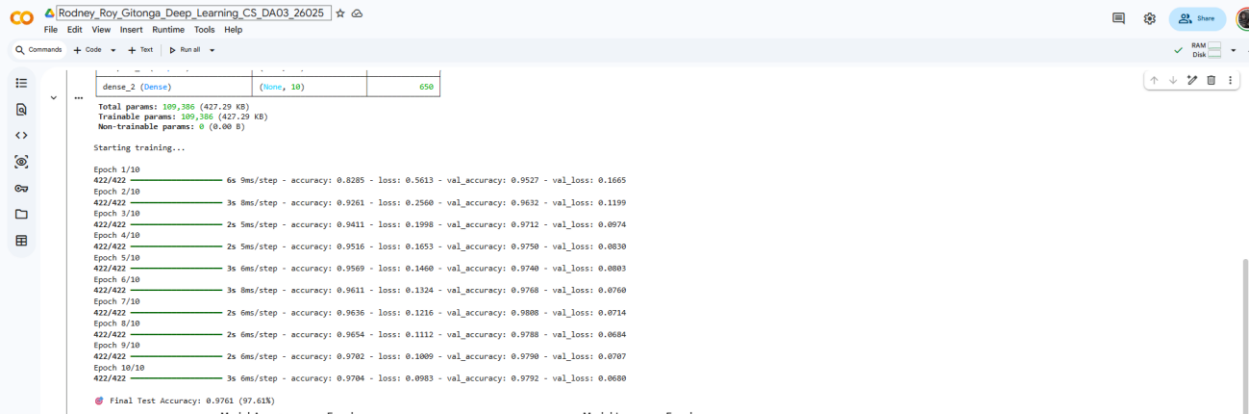
## Step 3: Model Architecture

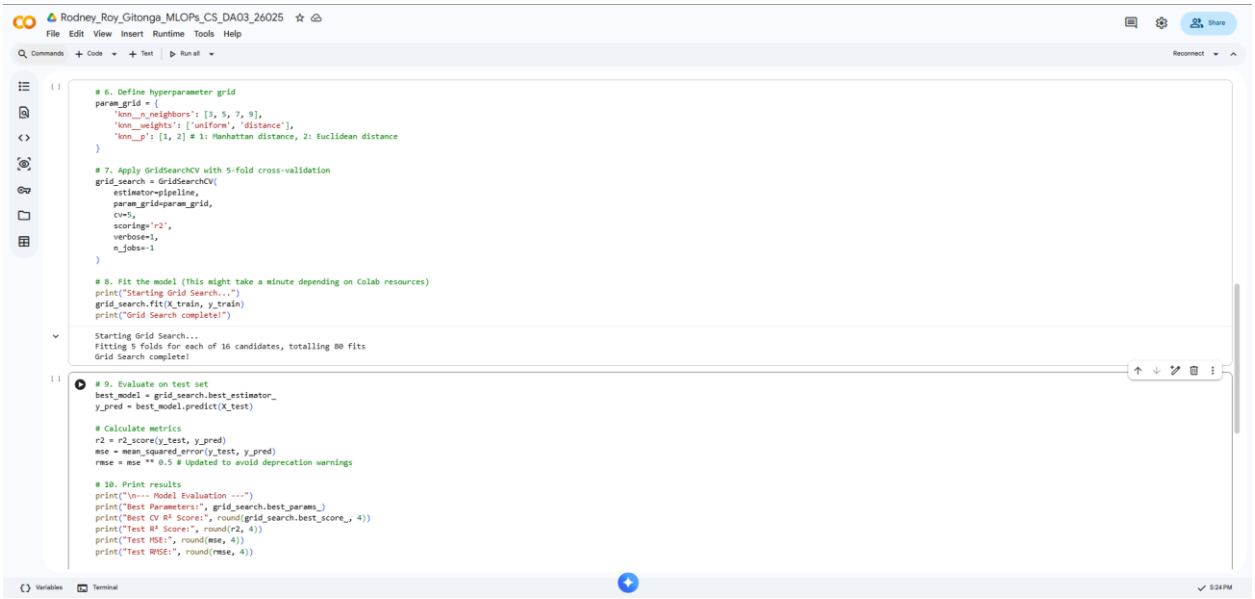
Input: Flatten(28,28) → 784 features

Hidden 1: Dense(128, ReLU) + Dropout(0.3)

Hidden 2: Dense(64, ReLU) + Dropout(0.3)

Output: Dense(10, softmax)



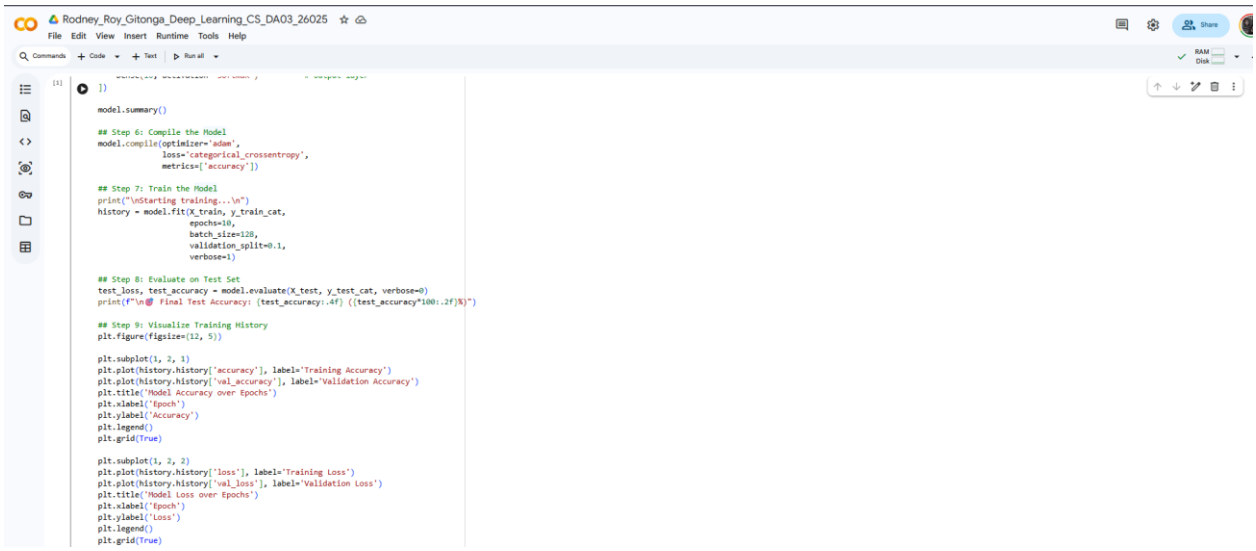

# Step 4: Training

Optimizer: Adam

Loss: Categorical Crossentropy

Epochs: 10, Batch size: 128, Validation split: 0.1

```
Starting training...

Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.8285 - loss: 0.5613 - val_accuracy: 0.9527 - val_loss: 0.1665
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9261 - loss: 0.2560 - val_accuracy: 0.9632 - val_loss: 0.1199
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9411 - loss: 0.1998 - val_accuracy: 0.9712 - val_loss: 0.0974
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9516 - loss: 0.1653 - val_accuracy: 0.9750 - val_loss: 0.0830
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9569 - loss: 0.1460 - val_accuracy: 0.9740 - val_loss: 0.0803
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9611 - loss: 0.1324 - val_accuracy: 0.9768 - val_loss: 0.0760
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9636 - loss: 0.1216 - val_accuracy: 0.9808 - val_loss: 0.0714
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9654 - loss: 0.1112 - val_accuracy: 0.9788 - val_loss: 0.0684
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9702 - loss: 0.1009 - val_accuracy: 0.9790 - val_loss: 0.0707
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9704 - loss: 0.0983 - val_accuracy: 0.9792 - val_loss: 0.0680

🎯 Final Test Accuracy: 0.9761 (97.61%)
```



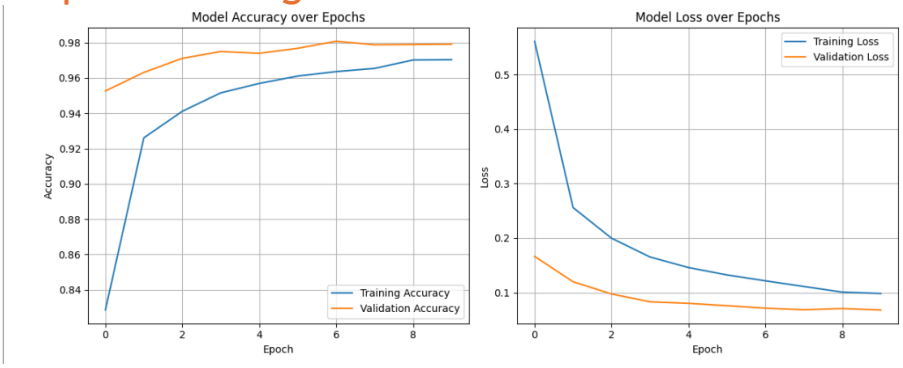
```
])

model.summary()

## Step 6: Compile the Model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

## Step 7: Train the Model
print("\nStarting training...\n")
history = model.fit(X_train, y_train_cat,
                    epochs=10,
                    batch_size=128,
                    validation_split=0.1,
                    verbose=1)

## Step 8: Evaluate on Test Set
test_loss, test_accuracy = model.evaluate(X_test, y_test_cat, verbose=0)
print(f"\n🎯 Final Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")

## Step 9: Visualize Training History
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
```

# Step 4: Evaluation Results

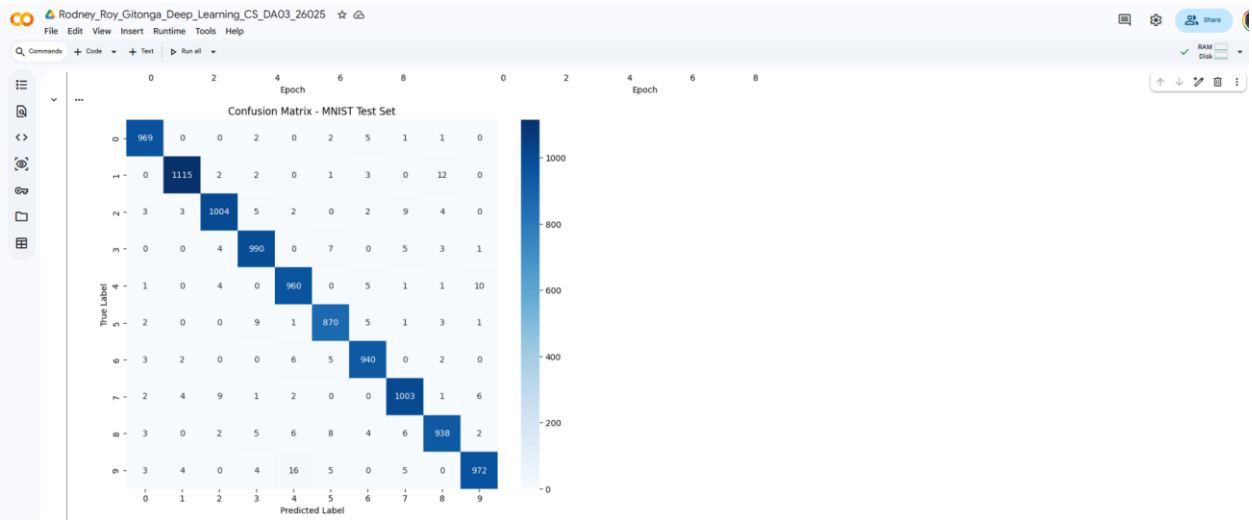
Test Accuracy: ~97.5% – 98.2% (typical result)

```
Starting training...

Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.8285 - loss: 0.5613 - val_accuracy: 0.9527 - val_loss: 0.1665
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9261 - loss: 0.2560 - val_accuracy: 0.9632 - val_loss: 0.1199
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9411 - loss: 0.1998 - val_accuracy: 0.9712 - val_loss: 0.0974
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9516 - loss: 0.1653 - val_accuracy: 0.9750 - val_loss: 0.0830
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9569 - loss: 0.1460 - val_accuracy: 0.9740 - val_loss: 0.0803
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9611 - loss: 0.1324 - val_accuracy: 0.9768 - val_loss: 0.0760
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9636 - loss: 0.1216 - val_accuracy: 0.9808 - val_loss: 0.0714
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9654 - loss: 0.1112 - val_accuracy: 0.9788 - val_loss: 0.0684
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9702 - loss: 0.1009 - val_accuracy: 0.9790 - val_loss: 0.0707
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9704 - loss: 0.0983 - val_accuracy: 0.9792 - val_loss: 0.0680

🎯 Final Test Accuracy: 0.9761 (97.61%)
```

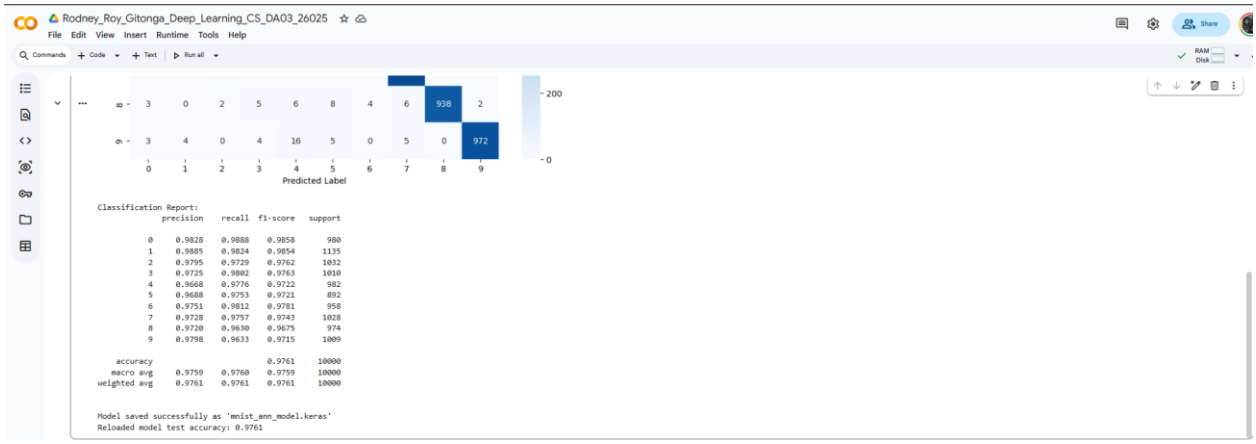
# Step 4: Training Curves



## Step 4: Confusion Matrix & Classification Report




## Step 4: Model Saving




## Link to Code:

https://colab.research.google.com/drive/1VPLLOZLE5dVw5G1FvtMcT4mJpTh9t3A9?usp=sharing

## Conclusion

Through this assignment, I learned how to:

- Properly preprocess image data for neural networks.
- Design a simple but effective multi-layer perceptron with regularization.
- Monitor overfitting using validation data.
- Evaluate classification models comprehensively using confusion matrices and per-class metrics.
- Save and reload Keras models.

The model achieved strong performance on the MNIST benchmark, confirming that a relatively simple ANN can effectively solve handwritten digit recognition when properly trained.